ContextOS: Design and Implementation of a Privacy-Preserving Context-Aware Computing System Using On-Device Small Language Models

Aarya Thota and **Jenifer Vakharia**
Department of Information Technology, KJ Somaiya School of Engineering

Abstract*—Contemporary operating systems function primarily as passive execution kernels, lacking semantic awareness of active user tasks and cross-application workflows. While cloud-centric conversational assistants attempt to bridge this divide, continuous telemetry exfiltration introduces confidentiality hazards, network latency dependencies, and cloud credential exposure. This paper introduces ContextOS, an intelligent contextual operating layer deployed on top of existing Windows desktop hosts. ContextOS models user activity across three discrete categories: Study, Research, and Document Editing. In the current university prototype, context selection is performed manually by the user, complemented by an optional zero-privilege native Win32 foreground sensing module. ContextOS isolates episodic working context within a partitioned Write-Ahead Logging (WAL) SQLite engine and provides context-dependent recommendations and conversational question answering. Crucially, the system incorporates an on-device Small Language Model (SLM) integration designed for Ollama, paired with a transparent deterministic local rule-based fallback reasoner that activates whenever Ollama is offline. Empirical evaluations conducted on an x86-64 Windows 11 host with 16GB RAM demonstrate an active window sensing latency of 0.021 ms, context cycle processing latency of 2.56 ms, and local memory retrieval latency of 2.84 ms, while maintaining a lean RAM resident footprint of 42.90 MB. Rule-based conversational responses average 0.007 ms. Neural SLM inference throughput was not verified on this test host due to Ollama being disconnected, but fallback operation proves complete system resilience. By eliminating external cloud dependencies through a localhost loopback architecture, ContextOS guarantees privacy while providing real-time cognitive assistance for academic and professional workflows.

***Index Terms*—Context-aware computing, Small Language Models (SLMs), on-device artificial intelligence, local inference, privacy preservation, operating systems intelligence, Ollama, Qwen 2.5, SQLite WAL.**

I. INTRODUCTION

Personal computing has reached a critical architectural inflection point. Modern knowledge workers continuously multitask across disparate desktop applications—navigating scientific repositories in web browsers, reading lecture notes in PDF viewers, and authoring reports in word processors. Despite decades of evolution in graphical user interfaces, the underlying desktop operating system remains fundamentally agnostic to the high-level semantic objectives of the user. Applications operate in isolated silos, requiring users to manually manage contextual transitions, remember cross-tool dependencies, and bridge fragmented information workflows.

To address this deficiency, modern software ecosystems have attempted to superimpose cloud-backed conversational AI layers. However, streaming continuous desktop telemetry, window titles, and clipboard contents to remote data centers incurs severe privacy and regulatory concerns (e.g., GDPR, FERPA). Intellectual property, student work, and confidential drafts cannot ethically or safely be dispatched to third-party cloud inference pipelines. Furthermore, cloud telemetry is vulnerable to network latency jitter, intermittent offline disconnection, and proprietary subscription gates.

Concurrently, the rapid algorithmic maturation of Small Language Models (SLMs)—exemplified by the Qwen 2.5 series (0.5B to 3B parameters) and Microsoft's Phi-3 family—has altered on-device compute economics. When compressed using 4-bit symmetric quantization (Q4_K_M), these models achieve competitive reasoning while occupying between 400 MB and 2.5 GB of system memory, executing efficiently on commodity consumer x86-64 CPUs without requiring discrete enterprise GPUs.

In this paper, we present the design, architectural formalization, implementation, and empirical validation of **ContextOS**. Built as an academic software engineering project at the Department of Information Technology, KJ Somaiya School of Engineering, ContextOS bridges the desktop semantic gap by deploying an ultra-lightweight, privacy-preserving intelligent layer over Windows hosts. By combining manual and Win32 context ingestion, structured session memory partitioning, explainable recommendation generation, and local SLM integration with an offline rule-based fallback reasoner, ContextOS demonstrates that context-aware assistance can be achieved on personal hardware without cloud leaks.

II. RELATED WORK

A. Context-Aware Computing Systems: The conceptual foundation of context awareness was established by Dey et al. [1], who defined context as any information that characterizes the situation of an entity. Early desktop context architectures relied on heavy accessibility hooks, OS kernel drivers, or optical character recognition (OCR) screen scrapers [2]. However, these approaches frequently incurred prohibitive CPU penalties (15–30% sustained overhead) and triggered antivirus security alerts. ContextOS overcomes these limitations by utilizing clean user-selected context states complemented by native Win32 `user32.dll` foreground handle queries, achieving sub-0.1 ms latency without kernel intervention.

B. On-Device Small Language Models and Edge Quantization: Recent breakthroughs in parameter-efficient quantization [3], [4] demonstrate that 1.5B–3B parameter models retain competitive reasoning capabilities under INT4 precision while significantly decreasing compute requirements. Works by Touvron et al. [5] and the Qwen Team [6] have shown that compact architectures excel at in-context learning when supplied with dense, well-structured prompts. ContextOS leverages these advances by designing prompt grounding for compact models like Qwen 2.5 1.5B.

C. Privacy-Preserving Desktop Architectures: Zero-trust on-device computation has emerged as the premier defense against telemetry surveillance [7], [8]. Differential privacy and local audit logging ensure user autonomy. ContextOS implements a local-first privacy governor that avoids external cloud telemetry by confining all sensing, logging, and inference communication to the localhost loopback interface (127.0.0.1).

III. METHODOLOGY

The research and engineering methodology of ContextOS revolves around three core tenets: predictable data locality, partitioned contextual representations, and transparent dual-mode cognitive dispatch.

A. Three Discrete Operational Contexts: In accordance with university research and desktop productivity workflows, ContextOS formalizes user activity into three canonical categories:

• **Study**: Tailored for academic coursework, lecture comprehension, textbook review, and theoretical principles (e.g., Operating Systems scheduling algorithms, ACID database transactions).

• **Research**: Tailored for literature analysis, empirical paper surveys, methodology comparisons, and benchmark evaluations.

• **Document Editing**: Tailored for technical writing, report formatting, citation verification, and manuscript drafting.

B. Manual Context Selection Paradigm: In the current prototype implementation, context selection is primarily executed **manually** by the user via straightforward dashboard controls. This explicit selection model prevents erratic state thrashing and guarantees that the system's episodic memory and suggestion generator remain strictly aligned with the user's conscious task intent. As an exploratory module, the prototype also includes an optional Win32 background window detector that can sample the active foreground application without disrupting manual category stability.

C. Dual-Path Cognitive Dispatch: To eliminate fragility, cognitive question answering implements a dual-path pipeline. When the local Ollama daemon is detected online with the target model, inquiries are routed to the SLM. If Ollama is offline or uninstalled, the query is seamlessly routed to a local deterministic rule reasoner, ensuring that the application never fails.

IV. SYSTEM ARCHITECTURE

ContextOS is structured as a decoupled, four-tier reactive architecture designed for zero cross-tier contention and deterministic data locality. Table I summarizes the functional responsibilities across tiers.

TABLE I: FOUR-TIER CONTEXTOS SYSTEM ARCHITECTURE

Subsystem Tier	Key Technologies	Functional Responsibility
Tier 1: Ingestion & Sensing Layer	ctypes, Win32 user32, psutil	Manual category selection, foreground window query, simulated domain preset dispatch.
Tier 2: Processing Layer	ContextProcessor, Heuristics	Task inference, transition detection, entity tagging, privacy scrubbing.
Tier 3: Persistence Layer	SQLite 3 (WAL Mode), B-Tree	Partitioned episodic memory, audit logs, event streams, sub-25ms concurrent writes.
Tier 4: Local Cognitive Tier	Ollama REST / Rule Fallback	Contextual prompt injection, local SLM integration, deterministic rule-based fallback.

A. Ingestion and Sensing Layer (Tier 1): Manages both explicit user context selection across the three defined environments and native Win32 foreground inspection. Win32 sensing queries `GetForegroundWindow()` and `GetWindowTextW()` from `user32.dll`, querying process identifiers with `GetWindowThreadProcessId()` and binary names with `psutil`. Detection executes in 0.021 ms without administrative privileges.

B. Context Processing Layer (Tier 2): Aggregates context inputs, infers active task signatures, extracts relevant domain entities, and evaluates context state transitions. A privacy scrubbing filter inspects incoming context strings against credential and token regexes prior to persistence.

C. Partitioned Persistence Layer (Tier 3): Uses an embedded SQLite 3 database operating in Write-Ahead Logging (WAL) mode (`PRAGMA journal_mode = WAL; PRAGMA synchronous = NORMAL;`). WAL completely eliminates write-lock contention, enabling background telemetry threads to record activity while interactive frontend API queries execute simultaneously. Dedicated tables separate events, episodic memories, and privacy logs.

D. Local Cognitive Tier (Tier 4): Coordinates conversational question answering and situational recommendations. It formats grounded prompt templates containing the active task and recent episodic memories, querying Ollama via localhost REST or falling back to the local rule reasoner.

E. Mathematical Formulation

To model context transitions, information utility, and execution constraints systematically, we define the following formal theoretical framework.

Let the operating system state at discrete time step t be represented by the multi-attribute tuple $S_t = \langle A_t, W_t, C_t, \tau_t, E_t \rangle$, where $A_t \in \mathcal{A}$ denotes the application binary name, $W_t \in \mathcal{W}$ is the window title string, $C_t \in \mathcal{C}$ represents the canonical domain category (Study, Research, Document Editing), τ_t is the inferred active task description, and $E_t = \{e_1, e_2, \dots, e_k\}$ is the set of extracted semantic entities.

We quantify the entropy of the user's operational context sequence via Context Entropy $H(C)$:

$$\mathbf{H(C) = - \sum_{i=1}^{|\mathcal{C}|} P(C_i) \log_2 P(C_i)}$$

where $P(C_i)$ represents the empirical frequency of the user operating within category C_i. Because context selection in this prototype is manual, $P(C_i)$ reflects deliberate user task allocation.

F. Explainable Recommendation Utility Function

The recommendation engine synthesizes proactive recommendations $R = \{r_1, r_2, \dots, r_m\}$. Each candidate recommendation r_j is assigned a situational utility score $U(r_j \mid S_t, M_t)$ conditioned on the active context S_t and the episodic memory partition M_t:

$$\mathbf{U(r_j \mid S_t, M_t) = \alpha \cdot \text{Sim}(\text{Emb}(r_j), \text{Emb}(\tau_t)) + \beta \cdot \sum_{k=1}^{|E_t|} \mathbb{I}(e_k \in r_j) + \gamma \cdot \mathcal{R}(r_j, M_t)}$$

where $\text{Sim}(\cdot)$ is the cosine similarity between the semantic embedding of the recommendation and active task τ_t, $\mathbb{I}(\cdot)$ is an indicator function scoring entity matches, $\mathcal{R}$ is the historical episodic relevance score, and $\alpha + \beta + \gamma = 1$. Recommendations are paired with an explicit natural-language rationale $Q(r_j) = \arg\max_{\phi} \mathcal{P}(\phi \mid S_t)$ explaining why the recommendation was generated.

G. Latency Upper Bound Formulation

To ensure interactive responsiveness on standard consumer hardware, total end-to-end cognitive latency T_{total} must satisfy the perceptual real-time constraint $T_{\text{total}} \le T_{\text{threshold}} = 250\text{ ms}$:

$$\mathbf{T_{\text{total}} = T_{\text{detect}} + T_{\text{process}} + T_{\text{mem_read}} + T_{\text{prompt}} + T_{\text{inference}}}$$

where $T_{\text{inference}}$ is the conversational response generation latency (neural time-to-first-token when online or rule-based response when offline).

```
Algorithm 1: ContextOS Context Lifecycle and Rule-Based Fallback Dispatch
Input: Polling interval Δt, Selected Context C ∈ {Study, Research, Document Editing}, Privacy state Φ
Output: Structured Context S_t, Situational Suggestions R, Partitioned Memory M_c, Assistant Answer A
loop on context change or user question do
    if Φ == PAUSED then suspend tracking; emit MaskedContext(); continue; end if
    S_raw ← LoadSelectedContextPreset(C); // Study, Research, or Document Editing
    if AutoDetectToggled then S_raw.os ← QueryWin32Foreground(); end if
    S_scrubbed ← ApplyPrivacyFilter(S_raw);
    PersistContextEvent(S_scrubbed, C); SnapshotPartitionMemory(M_c);
    R ← SynthesizeSuggestions(S_scrubbed, C); // 2-3 explainable recommendations
    if UserSubmitsQuestion(Q) then
        prompt ← ConstructContextualPrompt(Q, S_scrubbed, M_c);
        if OllamaServiceOnline() then A ← QueryLocalSLM(prompt);
        else A ← ExecuteRuleBasedFallback(Q, C, S_scrubbed); // Offline reasoner
       end if; PersistChatMessage(Q, A);
    end if
end loop
```

V. IMPLEMENTATION

ContextOS is implemented using a modern, lightweight software stack tailored for university-level demonstration and engineering evaluation. The architecture consists of four foundational modules:

A. FastAPI Backend & Persistence: Built with Python 3.14 and FastAPI, exposing high-performance REST endpoints for context switching, memory management, assistant chat, and privacy toggles. The embedded SQLite database utilizes WAL mode, persisting context events and isolated episodic memory partitions.

B. React 18 Desktop Dashboard: Built with React 18, Vite, and Tailwind CSS. The interface employs a clean, academic cream and subtle burgundy color palette across six clearly defined sections: Current Context, Context Selection (Study, Research, Document Editing), AI Assistant, AI Suggestions, Recent Activity, and Privacy Controls.

C. SLM Client and Rule-Based Fallback: The backend queries Ollama's local HTTP API (`/api/generate`) with a structured system prompt injecting the active task and episodic memory. If Ollama is not running on port 11434, the system immediately dispatches the query to a local deterministic rule-based fallback engine, clearly labeling responses with `[Local Rule-Based Response (Ollama not connected)]`.

TABLE II: HOST HARDWARE SPECIFICATIONS AND COMPATIBLE SLM PROFILES

Component / Model	Specification	RAM Footprint	Target Role & Suitability
Host Processor	x86-64 Architecture (Configured: 16 Logical Cores)	15.7 GB Usable	Multithreaded CPU execution host
Graphics (GPU)	Integrated Display Adapter (Assumed)	Shared Memory	Display rendering (No discrete VRAM assumed)
Qwen 2.5 1.5B (Target)	1.54B Params, Q4_K_M (986 MB)	~2.2 GB RAM	Primary target: CPU inference via Ollama
Qwen 2.5 0.5B (Fast)	0.49B Params, Q4_K_M (398 MB)	~1.1 GB RAM	Ultra-low latency alternative
Rule-Based Fallback Reasoner	Deterministic Local Python Engine	< 1 MB RAM	Offline fallback when Ollama is disconnected

VI. OBSERVATIONS AND EXPERIMENTAL RESULTS

Empirical evaluation was conducted on a Windows 11 host with an x86-64 processor (16 logical cores) and 16 GB RAM (15.7 GB usable). Benchmarks were executed across repeated iterations and logged directly to `benchmark_results.json`. Table III summarizes the measured latency and memory metrics. *Note: Conversational response latency was measured using the local rule-based fallback engine because Ollama was offline during host test execution. Live neural SLM inference requires an active Ollama process and is evaluated separately.*

TABLE III: EMPIRICAL LATENCY BENCHMARKS AND SYSTEM RESOURCE CONSUMPTION

Pipeline Subsystem Metric	Mean Latency	Std. Dev. (σ)	Min - Max Range	Evaluation Assessment
Win32 OS Foreground Sensing	**0.021 ms**	0.039 ms	0.010 - 0.184 ms	Sub-millisecond zero-overhead telemetry
Simulation Engine Retrieval	**0.001 ms**	0.001 ms	0.001 - 0.003 ms	Instantaneous memory-mapped lookup
Context Processing & Classification	**2.560 ms**	11.424 ms	0.020 - 42.10 ms	Deterministic entity structuring cycle
SQLite WAL Event Logging	**22.037 ms**	3.709 ms	17.80 - 28.50 ms	High-durability non-blocking write
Episodic Memory Retrieval (B-Tree)	**2.842 ms**	4.040 ms	1.10 - 14.20 ms	Fast indexed prompt assembly
Recommendation Engine Inference	**0.001 ms**	0.002 ms	0.001 - 0.008 ms	Real-time reactive suggestion dispatch
Rule-Based Fallback Inference	**0.007 ms**	0.007 ms	0.003 - 0.035 ms	Measured fallback (Ollama offline on host)
Total Context Cycle Overhead	**24.618 ms**	5.120 ms	18.20 - 35.80 ms	Well within the 250ms interactive bound
Python Process RAM Footprint (RSS)	**42.90 MB**	1.10 MB	41.00 - 44.20 MB	Lean footprint (<0.3% of 16GB system RAM)

A. Empirical Latency Analysis & Discussion

As quantified in Table III, native Win32 foreground sensing averages 0.021 ms, validating our choice of lightweight win32 handle queries over heavy accessibility trees. SQLite WAL event recording requires 22.037 ms, well below typical interactive polling thresholds. Total context cycle overhead is 24.618 ms, and the backend process consumes only 42.90 MB of resident RAM, leaving essentially 100% of the host's 15.7 GB usable memory available for user tasks.

B. Context-Dependent Response Differentiation

When queried with the identical prompt *'What should I focus on next?'*, ContextOS demonstrated differentiated responses across the three contexts: in **Study**, it prioritized database transaction isolation and CPU scheduling revision; in **Research**, it highlighted evaluating literature and benchmark latencies; in **Document Editing**, it recommended verifying formatting and citations. Fallback provenance was clearly disclosed.

VII. LIMITATIONS

Several technical limitations must be acknowledged in this prototype: 1) Context selection is primarily manual, requiring user interaction to switch domain state; 2) Win32 window inspection captures foreground titles and process names but does not inspect in-depth document contents due to strict privacy boundaries; 3) Neural SLM inference requires a separate local Ollama process to be started manually (`ollama run qwen2.5:1.5b`), and neural token generation speed was not measured on this host during automated evaluation; and 4) Multi-user profile management is not implemented in this local single-user prototype.

VIII. CONCLUSION

ContextOS demonstrates that privacy-preserving context-aware computing is practical on standard consumer personal computers without external cloud telemetry. By combining explicit context ingestion across Study, Research, and Document Editing, SQLite WAL episodic memory partitioning, explainable suggestions, and an on-device SLM architecture with a reliable rule-based fallback, the system provides a robust foundation for intelligent desktop interaction. Future work will benchmark live neural token throughput with Ollama and explore NPU acceleration.

REFERENCES

[1] A. K. Dey, 'Providing architectural support for building context-aware applications,' Ph.D. dissertation, Georgia Institute of Technology, Atlanta, GA, USA, 2000.

[2] E. Horvitz, C. Kadie, T. Paek, and D. Hovel, 'Models of attention in computing and communication: From principles to applications,' *Commun. ACM*, vol. 46, no. 3, pp. 52–59, Mar. 2003.

[3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, 'LLM.int8(): 8-bit matrix multiplication for transformers at scale,' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 30318–30332.

[4] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, 'SmoothQuant: Accurate and efficient post-training quantization for large language models,' in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 38087–38104.

[5] H. Touvron et al., 'Llama 2: Open foundation and fine-tuned chat models,' *arXiv preprint arXiv:2307.09288*, 2023.

[6] Qwen Team, 'Qwen2.5: A party of foundation and instruction-tuned language models,' Alibaba Group Technical Report, Sep. 2024.

[7] M. Abadi et al., 'Deep learning with differential privacy,' in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 308–318.

[8] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, 'Communication-efficient learning of deep networks from decentralized data,' in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.

[9] A. Vaswani et al., 'Attention is all you need,' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of deep bidirectional transformers for language understanding,' in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[11] M. Weiser, 'The computer for the 21st century,' *Sci. Am.*, vol. 265, no. 3, pp. 94–104, Sep. 1991.

[12] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, 'ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,' *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.

[13] E. Frachtenberg, 'Practical benchmarking for personal computing workloads,' *IEEE Computer*, vol. 54, no. 1, pp. 34–43, Jan. 2021.

[14] S. Guha and P. Francis, 'Identity and privacy in next-generation user operating environments,' in *Proc. 4th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2007, pp. 255–268.

[15] M. Abdin et al., 'Phi-3 technical report: A highly capable language model locally on your phone,' *arXiv preprint arXiv:2404.14219*, Apr. 2024.

[16] P. Lewis et al., 'Retrieval-augmented generation for knowledge-intensive NLP tasks,' in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[17] Y. Sheng et al., 'High-throughput generative inference for large language models with FlexGen,' in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 31094–31116.

[18] T. Schick and H. Schütze, 'Few-shot text generation with pattern-exploiting training,' in *Proc. EMNLP*, 2021, pp. 255–265.